06T: Transformations

Stat 230 - Spring 2026

> **Note**
>
> This is the tutorial version of the R activity. If you prefer to use .qmd and RStudio directly, you can use https://aluby.github.io/stat230/activities/06-transformations

1 Example: Mammal brain weights

Researchers wish to understand the relationship between the brain weights (y, in grams) and body weights (x, in kilograms) of mammals. They have data on a sample of 30 species of mammals, which can be loaded using the code shown below:

```
library(ggformula)
library(ggResidpanel)
weight <-  read.csv("https://aluby.github.io/stat230/data/weights.csv")
head(weight)
```

(a) Fit a simple linear regression model using body weight to predict brain weight.

```
#| min-lines: 3
```

(b) Create a scatterplot of y versus x with a regression line, a plot of the residuals vs. predicted (or "fitted") values ($\hat{y}$), and either a normal Q-Q plot or a histogram of the residuals. (Remember that you can use the `resid_panel()` command in the `ggResidpanel` R package for residual plots.) Summarize what issues you see with the regression conditions.

```
#| min-lines: 8
```

(b) Try various transformations of the explanatory and response variables to create a better linear regression model. (Hint: Notice that both the x and y variables are right skewed and have outliers, both may need a transformation.) What transformation(s) seem to remedy the issues you saw in part (b)?

```
#| min-lines: 8
```

2 Back-transforming your model

In Example 1 you selected transformation(s) to help "linearize" the relationship between brain weights and body weights. While this is helpful from a statistical perspective, it's often preferred to plot the fitted model on the **original scale** of the data. To do this, we need to back-transform the model using the following steps:

- Start by creating a scatterplot on the original scale.
- Add a `gf_lm()` layer, specifying the transformations in the `formula` and **if y is transformed** how to back-transform y via the `backtrans` argument.

For example, if we squared y, then we pass `y^2 ~ x` in as the formula to `gf_lm()` and `backtrans = sqrt` to back-transform y. Below is the full call where `df` is the data set with columns `xvar` and `yvar`.

```
# change variable names and data set name here
gf_point(yvar^2 ~ xvar, data = df) |>
  gf_lm(formula = y^2 ~ x , backtrans = sqrt, interval = "confidence")
```

Use this idea to plot the fitted model relating brain weights and body weights on the original scale.

```
#| min-lines: 3
```